第三章 存储系统

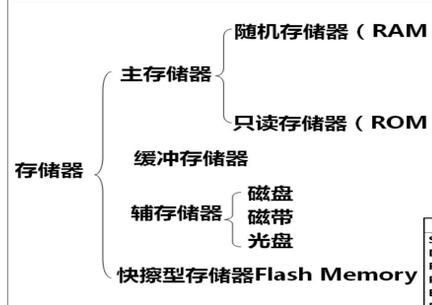
2022年9月10日 星期六 18:38

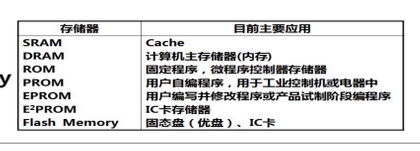
存储器的分类

按在计算机中的作用分类	主存储器【主存】	• 容量较小 • 存取速度较快 • 价格较高	• 用来存放计算机运行期间所需的程序和数据
	辅助存储器【辅存/外存】	• 容量大 • 存取速度较慢 • 单位成本低	• 用来存放当前暂时不用的程序和数据以及一些需要永久性保存的信息 • 辅存的内容需要调入主存后才能被CPU访问
	高速缓存存储器【Cache】	• 速度可与CPU匹配 • 存储容量小 • 价格高	• 位于主存和CPU之间 • 用来存放当前CPU经常使用的指令和数据，以便CPU能高速地访问他们 • 现代计算机通常将其制作在CPU中
按存储介质分类	• 磁表面存储器：磁盘，磁盘 • 磁芯存储器 • 半导体存储器：MOS型存储器，双极型存储器 • 光存储器：光盘		
按存取方式分类	随机存储器【RAM】	• 读写方便，使用灵活 • RAM分为静态RAM和动态RAM • RAM主要为用户编程设置的	• 存储器的任何一个存储单元都可以随机存取 • 存取时间与存储单元的物理位置无关 • 主要用于做主存或高速缓冲存储器
	只读存储器【ROM】	• ROM与RAM一起统一构成主存的地址域 • ROM和RAM的存取方式均为随机存取 • 操作系统的内存储器既有RAM也有ROM • 广义上的ROM现在可通过电擦除进行写入 • ROM存放系统程序，标准子程序和各类常数	• 存储器的内容只能随机读出而不能写入 • 信息一旦写入就不变，断电后也不消失 • 通常用于存放固定不变的程序，常数和汉字字库
	串行访问存储器	顺序存取存储器【磁带】 • 存取速度慢 • 只能按某种顺序存取	直接存取存储器【磁盘，光盘】 • 既不是随机存取也不是按顺序存取

按信息可保持性分类	• 易失性存储器
	  ○ 断电后，存储信息消失【如RAM】
	• 非易失性存储器
	  ○ 断电后，信息仍保存【如ROM，磁表面存储器，光存储器】		

总结


存储器
主存储器
随机存储器（RAM）
静态RAM(SRAM)
动态RAM(DRAM)
只读存储器（ROM）
掩模式ROM
可编程式PROM
可擦写式EPROM
电擦写式EEPROM
缓冲存储器
辅存储器
磁盘
磁带
光盘
快擦型存储器Flash Memory



存储器	目前主要应用
SRAM	Cache
DRAM	计算机主存储器(内存)
ROM	固定程序，微程序控制器存储器
PROM	用户自编程序，用于工业控制机或电器中
EPROM	用户编写并修改程序或产品试制阶段编程序
E²PROM	IC卡存储器
Flash Memory	固态盘（优盘）、IC卡

其他考点

- 主存储器即主存，存储指令和数据，由RAM和ROM实现
- 主存储器在CPU外，按地址访问
- 控制存储器用来存放实现指令系统的所有微指令，由ROM实现
- 控制存储器在CPU的控制器内，按照微指令的地址访问

存储器的性能指标

存储容量	• 存储容量 = 存储字数 * 字长（如1M*8bit） • 存储字数表示存储器的地址空间大小【MAR】 • 字长表示一次存取操作的数据量【MDR】	• 1G=1024M；1M=1024K • $1s = 10^3 ms$【毫秒】$= 10^6 us$【微秒】$= 10^9 ns$【纳秒】$= 10^{12} ps$【皮秒】
单位成本	• 每位价格 = 总成本 / 总容量	
存取速度	• 数据传输率 = 数据的宽度/ 存储周期 • 数据传输率中的K,M是10的次方不是2 的次方，只有存储容量是二的次方 • 存储周期大于存取时间，因为有一段恢复内部状态的复原时间	• 存取时间T_a = 从启动一次存储器到完成该操作所经历的时间 • 存取周期T_m = 存储器进行一次完整的读写操作所需的全部时间 • 主存带宽B_m = 数据传输率 = 每秒从主存进出信息的最大数量 = 单位$\frac{字}{秒}$

多级层次的存储系统

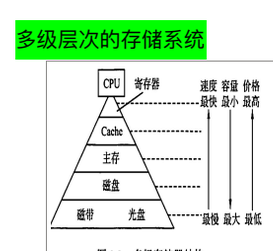

CPU
寄存器
Cache
主存
磁盘
磁带
光盘
速度 容量 价格
最快 最小 最高
最慢 最大 最低

图 3.2 多级存储器结构

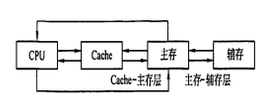

CPU
Cache
主存
辅存
Cache-主存层
主存-辅存层

图 3.3 三级存储系统的层次结构及其构成

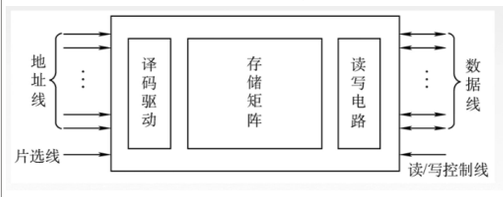
- 层次结构：Cache---主存层和主存---辅存
- 前者解决CPU和主存速度不一致的问题
- 后者解决存储系统的容量问题
- 层次思想：上一层的存储器作为低一层存储器的高速缓存，上一层的内容是下一层的的内容的一部分
- Cache和主存能与CPU直接交换信息；辅存要通过主存与CPU交换信息；
- 主存与CPU，Cache，辅存都能交换信息
- 主存和Cache之间的数据调动是由硬件自动完成的，对所有程序员透明
- 主存和辅存之间的数据调动是由操作和os共同完成的【换入换出技术】，对应用程序员透明

主存储器

【存储器芯片的内部结构】&【主存储器的内部结构】

存储器芯片的组成

- 存储体+IO/读写电路+地址译码器+片选控制信号+读写控制信号
- 读RD/写WR控制线：决定芯片是进行读还是写操作
- 片选线CS：确定哪个存储芯片被选中。可用于容量扩充
- 引脚最低数目 = 片选线(1) + 控制线(2, 读RD写WR) + 数据线 + 地址线


地址线
译码驱动
存储矩阵
读写电路
数据线
片选线
读/写控制线

主存储器的组成部分

- 数据线的宽度 = MDR的宽度 = 存储字长
- 地址线的宽度 = MAR的宽度 = 存储字数
- 下图总容量 = 2^{36}×64位 = 2^{39}B

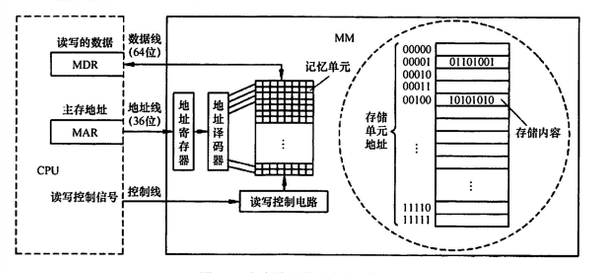
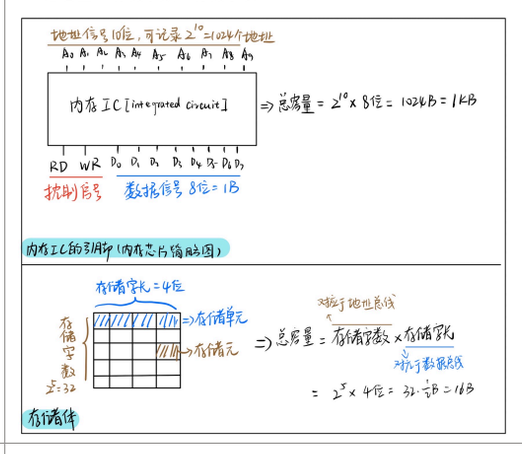

读写的数据
MDR
数据线(64位)
主存地址
MAR
地址线(36位)
CPU
读写控制信号
控制线
地址寄存器
地址译码器
记忆单元
MM
读写控制电路
存储单元地址
存储内容
00000
00001
00010
00011
00100
11110
11111
01101001
10101010

图 3.6 主存储器的基本组成框图

存储体的组成元素

- 存储体【房子】+存储单元【房间】+存储元【房间里面的床】
- 总容量 = 存储字数 * 存储字长


地址信号10位，可记录2^{10}=1024个地址
A₀ A₁ A₂ A₃ A₄ A₅ A₆ A₇ A₈ A₉
内存IC[integrated circuit]
⇒总容量 = 2^{10} × 8位 = 1024B = 1KB
RD WR D₀ D₁ D₂ D₃ D₄ D₅ D₆ D₇
控制信号
数据信号 8位 = 1B
内存IC的引脚（内存芯片简略图）
存储字长 = 4位
存储单元
存储元
存储字数 2^5=32
⇒总容量 = 存储字数 × 存储字长
对应于地址总线
对应数据总线
= 2^5 × 4位 = 32·4b = 16B
存储体

寻址方式

- 题目上没有明确指定按字编址，那么就默认是按字节编址【一字节8位】
- 32位的计算机中：32位(bit)=4字节(byte)=1字(word)
- 64位的计算机中：64位(bit)=8字节(byte)=1字(word)
- 存放一个机器字的存储单元，通常称为字存储单元，相应的单元地址叫字地址。
- 存放一个字节的存储单元，称为字节存储单元，相应的地址称为字节地址。
- 如果计算机中可编程的最小单位是字存储单元，则该计算机称为按字寻址的计算机。
- 如果计算机中可编程的最小单位是字节，则该计算机称为按字节寻址的计算机。
- 一个机器字可以包含数个字节，所以一个存储单元也可以包含数个能够单独编制的字节地址

例如，一个16位二进制的字存储单元可存放两个字节，可以按字编址，也可以按字节编址
当用字节编址时，16位的存储单元占两个字节地址。

地址复用技术

- 由于DRAM芯片容量大，地址位数多，为了减少地址引脚线，采用地址复用技术
- DRAM因为分两次发送，长度相同，因此地址线可以复用，线数减少了一半
- 引脚数 = 地址线减半 + 数据线不变 + 行通选(1) + 列通选(1) + 读写控制线(2)
- 片选线用行通选线替代↑
- DRAM采用地址复用技术【计算引脚时不要忘记了】，SRAM不采用

SRAM和DRAM的比较

	SRAM【*Static Random Access Memory*】	DRAM【*Dynamic Random Access Memory*】
主要用途	Cache	主机内存
存储信息	触发器，双稳态电路	电容
破坏性读出	非	是
需要刷新	不要	需要
送行列地址	同时送	分两次送
运行速度	快	慢
集成度	低	高
存储成本	高	低

只读存储器【ROM】

ROM的特点	• 结构简单，位密度比可读写存储器高 • 具有非易失性，可靠性高
ROM 的类型	【掩模式只读存储器 MROM】 • 内容在生产过程中写入。可靠性高，集成度高，价格便宜但灵活性差 【一次可编程只读存储器 PROM】 • 用于用户实现一次性编程。 【可擦除可编程只读存储器 EPROM】 • 用于用户实现多次性编程。可多次改写，但次数有限，写入时间过长。 【Flash存储器】 • 既可在不加电的情况下长期保存信息，又能在线进行快速擦除和重写 • U盘就是基于Flash的只读存储器 • 价格便宜，集成度高，电可擦除重写且擦除重写速度快。 【固态硬盘 SSD】 • SSD即闪存，是一种非易失性存储器，采用随机访问方式 • 可长期保存信息，快速擦除和重写，相比传统硬盘也有读写速度快、低功耗的特性，但价格较高。

DRAM的刷新

常考知识点

什么是刷新?

- DRAM电容的电荷维持时间短，即使电源不断电，信息也会自动消失
- 因此每隔一段时间必须刷新，一般取2ms，即刷新周期（再生周期）
- DRAM的刷新是以行为单位的
- 一次完整的刷新过程只需要占用一个存储周期

假设如下指标

- 刷新周期为2ms；存取周期为0.5μs，即刷新1行的时间为0.5μs；刷新时间=存取周期因为刷新的过程与一次存取相同，只是没有在总线上输入输出
- 存取周期＞真正用于存取的时间因为存取周期内、存取操作结束后仍然需要一些时间来更改状态
- 对128×128的矩阵的存储芯片进行刷新，按存储单元(1B/单元)分为128行128列，即128×128×1B/单元=2^14个单元×1B/单元 = 16KB内存
- 如果是64×64的矩阵，则为64×64×1B/单元=2^12个单元×1B/单元 = 4KB内存】

集中刷新

集中刷新

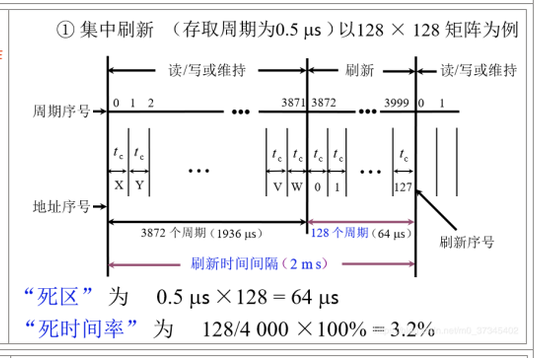
- 在规定的一个刷新周期内，对全部存储单元集中一段时间逐行进行刷新，此刻必须停止读/写操作

刷新过程

- 用0.5μs*128=64μs的时间对128行进行逐行刷新
- 由于这64μs的时间不能进行读/写操作，故称为“死时间”或访存“死区”
- 由于存取周期为0.5μs，刷新周期为2ms，即4000个存取周期

为什么刷新与存取不能并行?

- 因为内存就一套地址译码和片选装置，刷新与存取有相似的过程
- 它要选中一行【这期间片选线、地址线、地址译码器全被占用着】
- 同理，刷新操作之间也不能并行【意味着一次只能刷一行】


① 集中刷新 （存取周期为0.5 μs）以128 × 128 矩阵为例
读/写或维持
刷新
读/写或维持
周期序号
地址序号
3872 个周期（1936 μs）
128 个周期（64 μs）
刷新时间间隔（2 ms）
刷新序号
“死区” 为 0.5 μs × 128 = 64 μs
“死时间率” 为 128/4 000 × 100% = 3.2%

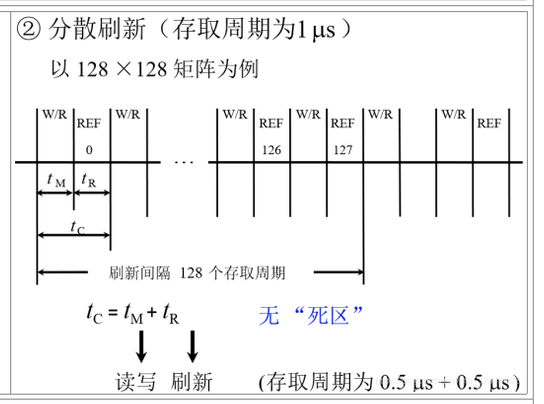
分散刷新

分散刷新

- 是指对每行存储单元的刷新分散到每个存取周期内完成
- 其中，把机器的存取周期tc分成两段，前半段tM用来读/写或维持信息，后半段tR用来刷新

刷新过程

- 在每个存取操作后绑定一个刷新操作，延长了存取周期。
- 这样存取周期就成了0.5μs + 0.5μs =1μs
- 但是由于与存取操作绑定，就不需要专门给出一段时间来刷新了
- 这样，每有128个读取操作，就会把0-127行全部刷新一遍
- 故每隔128μs 就可将存储芯片全部刷新一遍【即刷新周期是1μs×128=128μs远短于2ms】
- 而且不存在停止读/写的死时间，但是存取周期长了，整个系统速度降低了
- 分散刷新的刷新周期128μs ，其实不需要这么频繁，会导致浪费


② 分散刷新（存取周期为1 μs）
以 128 × 128 矩阵为例
W/R REF 0
W/R REF 126
W/R REF 127
刷新间隔 128 个存取周期
$t_C = t_M + t_R$
读写 刷新
无 “死区”
（存取周期为 0.5 μs + 0.5 μs）

异步刷新

异步刷新

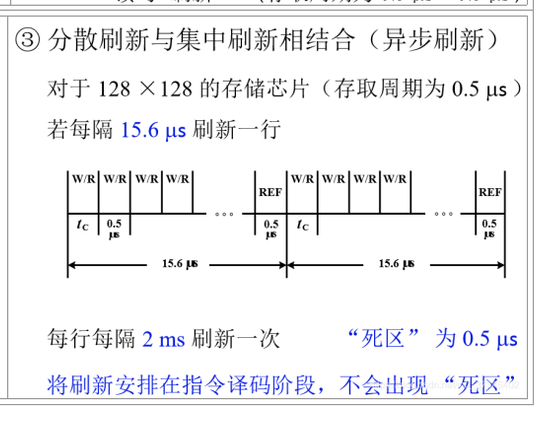
- 既可以缩短“死时间”【仍然存在死时间】，又充分利用最大刷新间隔为2ms的特点

刷新过程

- 具体操作为：在2ms内对128行各刷新一遍
- 即每隔15.6μs刷新一行(2000μs/128≈15.6μs)，而每行刷新的时间仍为0.5μs
- 这样，刷新一行只能停止一个存取周期
- 但对每行来说，刷新间隔时间仍为2ms，而死时间为0.5μs
- 相对每一段来说，是集中式刷新，相对整体来说，是分散式刷新

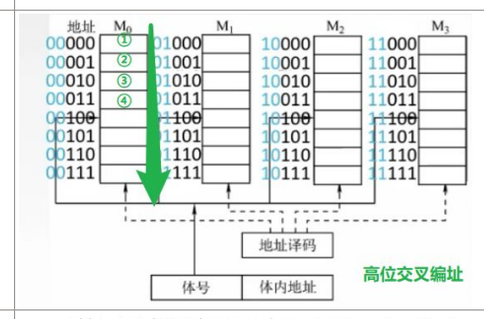
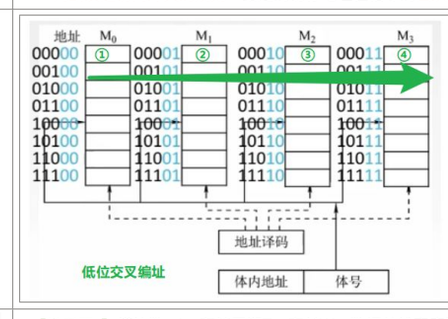
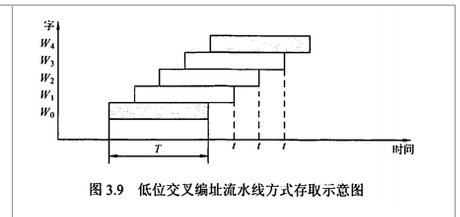
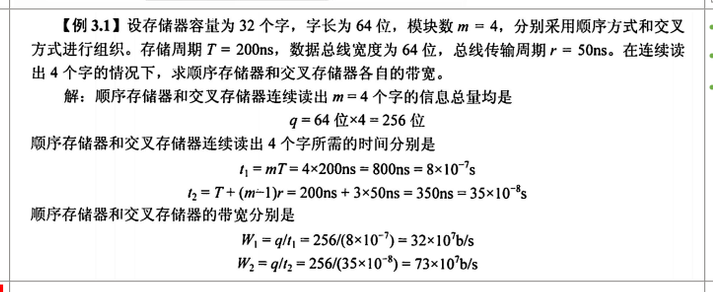
特点

- 如果将DRAM的刷新安排在CPU对指令的译码阶段
- 由于这个阶段CPU不访问存储器，所以这种方案既克服了分散刷新需独占0.5μs用于刷新
- 使存取周期加长且降低系统速度的缺点，又不会出现集中刷新的访存“死区”问题
- 从根本上提高了整机的工作效率


③ 分散刷新与集中刷新相结合（异步刷新）
对于 128 × 128 的存储芯片（存取周期为 0.5 μs）
若每隔 15.6 μs 刷新一行
W/R REF
15.6 μs
每行每隔 2 ms 刷新一次
“死区” 为 0.5 μs
将刷新安排在指令译码阶段，不会出现 “死区”

多模块存储器

常考点	• 多模块存储器：一种空间并行技术，利用多个结构完全相同的存储模块的并行工作来提高存储器的吞吐率
单体并行存储器	• 【定义】存储器中只有一个存储体，每个存储单元存储m个字，总线宽也为m个字，地址必须顺序排列并处于同一存储单元 • 【通俗理解】换一台多列的快递柜而不能理解为多个一列的快递柜放在一起 • 【过程】在一个存取周期内，从同一地址取出m条指令，然后将指令逐条送至CPU • 【缺点】指令和数据在主存内必须是连续存放的，一旦遇到转移指令，或操作数不能连续存放，这种方法的效果就不明显
多体并行存储器	• 【定义】由多体模块组成，每块都有相同容量和读取速度，各模块都有独立的读写控制电路、MAR和MDR。既能并行工作也能交叉工作。 • 【通俗理解】把多个单列的快递柜拼放在一起，通过加上一些控制电路把他们连接起来，这种叫多体并行存储器。

	高位交叉编址（顺序方式）【竖】	低位交叉编址（交叉方式）【横】
定义	【特点】先在一个模块内访问，等到该模块访问完之后才转到下一个模块访问 【编号】高位地址表示体号【模块号】，低位地址表示体内地址 【优点1】某个模块进行存取时，其它模块不工作 【优点2】某一模块出现故障时，其它模块可以照常工作 【优点3】通过增添模块来扩充存储器容量比较方便 【缺点】各模块串行工作，存储器的带宽受到了限制，并不能提高吞吐量	【特点1】连续地址分布在相邻的不同模块内，同一模块内的地址是不连续的 【特点2】地位交叉编制是交叉存放的，满足程序的局部性原理 【编号】高位地址表示体内地址，低位地址表示体号【模块号】 【优点】对连续字的成块传送可实现多模块并行存取，提高了存储器的带宽 【计算】 • 每个模块按“模m”交叉编址，模块号=单元地址%m • 设模块字长等于数据总线宽度，模块存取一个字的存取周期为T，总线传输周期为r • 则存储器交叉模块的数目最小为m=T/r • 每隔r时间延迟后启动下一模块，当数目不小于m时，就可以保证T时间之后再启动该模块，流水线就不会断 • 取m个字的时间为T+(m−1)r，顺序方式时间为mT • 判断发送访问冲突的规则：给定的访存地址在相邻的四次访问中出现在同一个存储模块中【m=4时】
示意图	 高位交叉编址	 低位交叉编址  图 3.9 低位交叉编址流水线方式存取示意图
【计算带宽】	 【例 3.1】设存储器容量为 32 个字，字长为 64 位，模块数 $m = 4$，分别采用顺序方式和交叉方式进行组织。存储周期 $T = 200ns$，数据总线宽度为 64 位，总线传输周期 $r = 50ns$。在连续读出 4 个字的情况下，求顺序存储器和交叉存储器各自的带宽。 解：顺序存储器和交叉存储器连续读出 $m = 4$ 个字的信息总量均是 $q = 64位×4 = 256位$ 顺序存储器和交叉存储器连续读出 4 个字所需的时间分别是 $t_1 = mT = 4×200ns = 800ns = 8×10^{-7}s$ $t_2 = T + (m−1)r = 200ns + 3×50ns = 350ns = 35×10^{-8}s$ 顺序存储器和交叉存储器的带宽分别是 $W_1 = q/t_1 = 256/(8×10^{-7}) = 32×10^7 b/s$ $W_2 = q/t_2 = 256/(35×10^{-8}) = 73×10^7 b/s$	• 【常考题1】模块数=4，存储周期T，字长W，数据总线宽度即为W，总线传输周期r，连续存取n个字，分别求带宽 • 【翻译】有m个存储体，存储周期为T，字长为W，每隔r时间启动下一个存储体，连续存取n个字，分别求带宽 • 【公式】带宽 = 存储速率 = $\frac{总量}{时间} = \frac{n×W}{t}$【顺序t = mT】【交叉t = T + (m − 1)r】
【流水线计算】		

地址
M₀ M₁ M₂ M₃
地址译码
体号
体内地址
00000 00001 00010 00011 00100 00101 00110 00111
01000 01001 01010 01011 01100 01101 01110 01111
10000 10001 10010 10011 10100 10101 10110 10111
11000 11001 11010 11011 11100 11101 11110 11111
字
W₀ W₁ W₂ W₃ W₄
T
时间

主存储器与CPU的连接

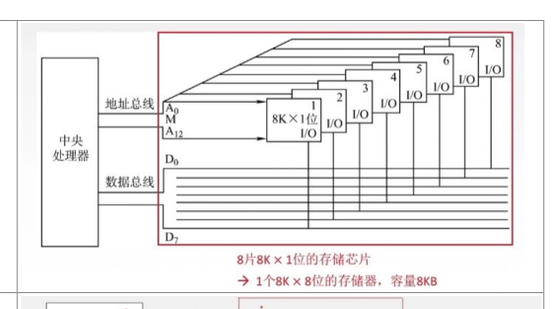
CPU与主存储器的连接原理

- CPU读指令，通过地址线去访问存储器的MAR（地址寄存器）
- MAR（地址寄存器）通过选通线去访问矩阵中的数据
- 矩阵需要通过数据线与MDR（数据寄存器）进行接发

主存容量的扩展

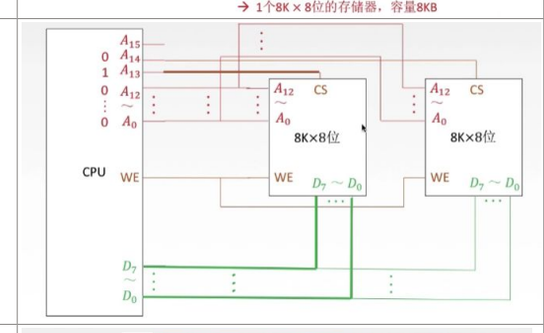
位扩展法

- 8K* 8位的存储器 = 8片 8K*1位的RAM组成
- 地址线并行，数据线一一接上


中央处理器
地址总线
数据总线
A₀ M A₁₂
D₀ D₇
8K×1位 I/O
1 2 3 4 5 6 7 8
8片8K × 1位的存储芯片
→ 1个8K × 8位的存储器，容量8KB

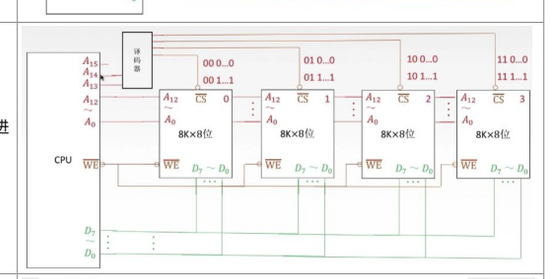
字扩展法【线选法实现】

- 16K * 8位的存储器 = 2片 8K*8位的存储器
- 右图有点问题，地址线应该是A0~A11共12位，译码线是A12和A13共2位
- 由片选信号来区分各芯片的地址范围
- 当A12为1时，第一块工作，A13的CS必须为0
- 谁工作，数据线就接送谁的数据，即将CS设置为1
- 2位二进制时：只能利用01，10；


CPU
A₁₅ A₁₄ A₁₃ A₁₂ ~ A₀
0 1 0 0
WE
D₇ ~ D₀
A₁₂ CS
8K×8位
WE D₇ ~ D₀

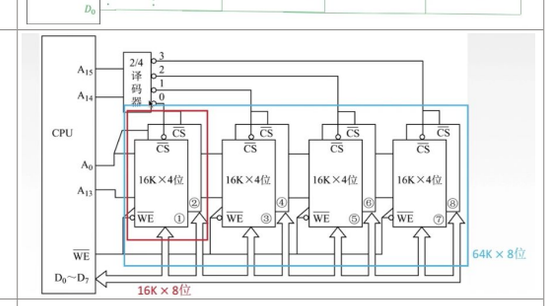
字扩展法【译码片实现】

- 有4块芯片，不需要4条线而只需要两条
- 右图有点问题，地址线应该是A0~A11共12位，译码线是A12和A13共2位
- 2位二进制时：可以利用00，01，10，11；
- 【译码器】一个二进制转十进制的物理元件，将左边三根地址线表示的二进制意义映射到右边十进制的选通线

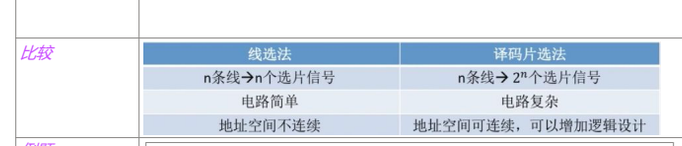

译码器
00 0…0 00 1…1
01 0…0 01 1…1
10 0…0 10 1…1
11 0…0 11 1…1
8K×8位
CPU
WE

字位同时扩展法

- 一块芯片只有4位，因此通过2片叠加先实现位拓展
- 等价于实现了一个8位的存储芯片
- 再通过译码片选的方式实现字拓展
- 在不同的地址线中选择不同的芯片组合进行工作


A₁₅ A₁₄
2/4 译码器
CPU
A₀ A₁₃
16K×4位
CS WE
① ② ③ ④ ⑤ ⑥ ⑦ ⑧
64K × 8位
16K × 8位
D₀~D₇

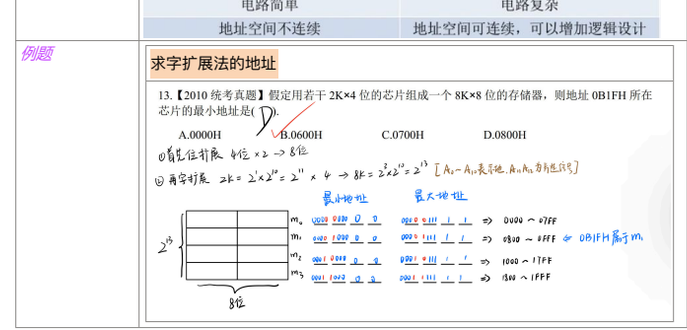
比较



线选法	译码片选法
n条线→n个选片信号	n条线→ 2^n个选片信号
电路简单	电路复杂
地址空间不连续	地址空间可连续，可以增加逻辑设计

例题

求字扩展法的地址



13.【2010 统考真题】假定用若干 2K×4 位的芯片组成一个 8K×8 位的存储器，则地址 0B1FH 所在芯片的最小地址是(D)。

A.0000H B.0600H C.0700H D.0800H

①首先位扩展 4位×2 → 8位

②再字扩展 2k = $2^1×2^{10} = 2^{11}$ × 4 → 8k = $2^3×2^{10} = 2^{13}$ [A₀~A₁₀表示地址, A₁₁A₁₂为片选信号]

最小地址 最大地址

m₀ 0000 0000 0 0 0000 0111 1 1 ⇒ 0000 ~ 07FF

m₁ 0000 1000 0 0 0000 1111 1 1 ⇒ 0800 ~ 0FFF ← 0B1FH属于m₁

m₂ 0001 0000 0 0 0001 0111 1 1 ⇒ 1000 ~ 17FF

m₃ 0001 1000 0 0 0001 1111 1 1 ⇒ 1800 ~ 1FFF

2^{13} 8位

外部存储器【磁盘见操作系统，操作系统磁盘讲的更详细】

固态硬盘SSD

- 【定义】是一种基于闪存技术的存储器
- 【组成】一个SSD由一个或多个闪存芯片【替代磁盘的机械驱动器】和闪存翻译层【扮演磁盘控制器的角色】组成
- 【优点】
  - 由半导体部件组成，没有移动的部件
  - 因为1，随机访问的时间比机械硬盘快很多
  - 没有机械噪声和震动
  - 能耗低
  - 抗震性好
- 【缺点】主要是容易磨损

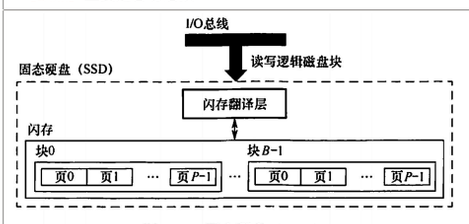
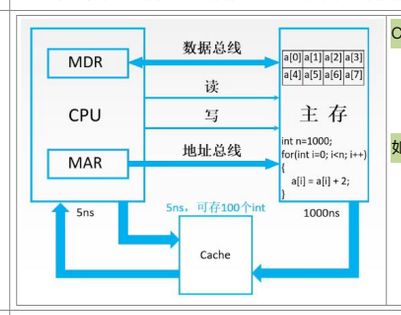

I/O总线
读写逻辑磁盘块
固态硬盘（SSD）
闪存翻译层
闪存
块0
块B-1
页0 页1 … 页P-1

图 3.15 固态硬盘（SSD）

高速缓冲存储器

什么是Cache?	• 高速缓冲存储器就是存在于主存与CPU之间的一级存储器，有了它CPU可以直接对其存取数据，从而减少了时间，提高了系统的运行速度 • CPU与Cache/主存的信息交互单位为字，Cache与主存的信息交互单位为块 • 一个块通常由若干字组成
为什么Cache能提升存储速度?	局部性原理 • 时间局部性是指如果程序中的某条指令一旦执行，则不久之后该指令可能再次被执行；如果某数据被访问，则不久之后该数据可能再次被访问【例如循环】 • 空间局部性是指一旦程序访问了某个存储单元，则不久之后。其附近的存储单元也将被访问【例如对数组的访问，如果数组按行存的，则先行再列的取方式空间局部性更好】 Cache利用了局部性原理 • 将程序中正在使用的部分存在在容量较小但速度更快的cache中，使CPU更多是针对它进行的来提升速度

Cache的读写过程


MDR
CPU
MAR
数据总线
读
写
地址总线
主 存
a[0] a[1] a[2] a[3] a[4] a[5] a[6] a[7]
int n=1000;
for(int i=0; i<n; i++)
a[i] = a[i] + 2;
5ns
5ns，可存100个int
1000ns
Cache

CPU、Cache、和主存三者的读写关系

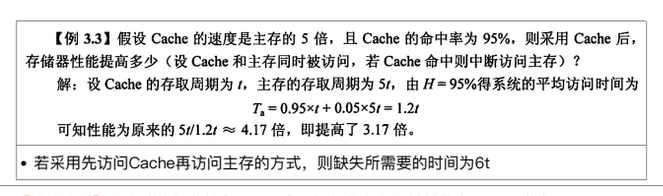
- Cache会从主存中一并读取目标数据以及附近空间的数据【通常以块为单元取出】【速度如图需要1000ns】
- CPU要取数据会优先从Cache中读取，因为速度快【速度如图5ns】
- CPU计算完后，会把数据再次返回给Cache【速度也是5ns】

如果无法从Cache中直接高速存取，CPU会用两种模式进行补救

- 赶紧让Cache读取主存，CPU再从Cache中读取。
- CPU一边直接访问主存读取，一边让Cache读取主存，CPU再从Cache中读

Cache的性能分析

- Cache命中率 $= \frac{Cache的总命中次数}{Cache的总命中次数 + 访问主存的总次数}$
- 系统平均访问时间 = 命中的概率×命中所需要花费的时间+缺失的概率×平均访存次数×一次总线读突发总线事务所需时间【13年统考大题】
- 性能效率 $= \frac{访问Cache的时间}{系统平均访问时间}$



【例 3.3】假设 Cache 的速度是主存的 5 倍，且 Cache 的命中率为 95%，则采用 Cache 后，存储器性能提高多少（设 Cache 和主存同时被访问，若 Cache 命中则中断访问主存）？

解：设 Cache 的存取周期为 t，主存的存取周期为 $5t$，由 $H = 95\%$得系统的平均访问时间为

$$T_a = 0.95×t + 0.05×5t = 1.2t$$

可知性能为原来的 $5t/1.2t ≈ 4.17$ 倍，即提高了 3.17 倍。

- 若采用先访问Cache再访问主存的方式，则缺失所需要的时间为6t

实现Cache要解决的关键问题

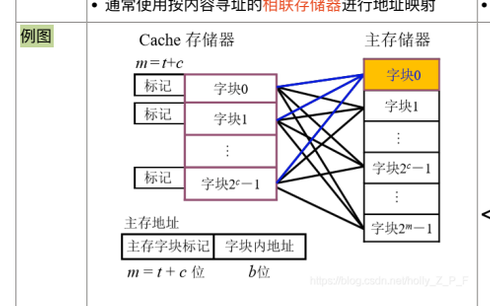
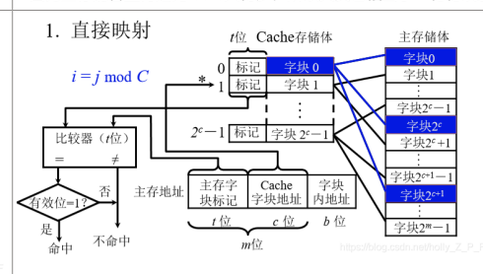
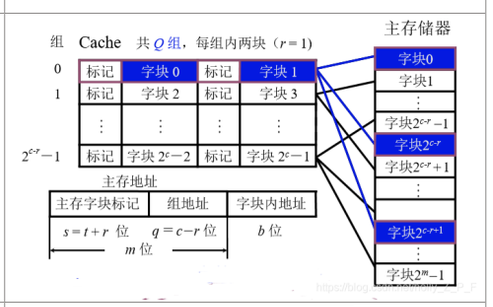
- 【地址映射】主存块如何存放在Cache中，如何将主存地址转换为Cache地址
- 【替换策略】Cache满后，使用何种策略对Cache块进行替换或淘汰
- 【写入/更新策略】如何既保证主存块和Cache块的数据一致性，又尽量提升效率

Cache相关知识考点

- 采用指令cache与数据cache分离的主要目的是减少指令流水线资源冲突

虚拟存储器【见操作系统，操作系统见的更详细】

【地址映射】解决主存中的块放到Cache中的哪个位置

	空位随意放——【全相联映射】	对号入座——【直接映射】	按号分组，组内任意放——【组相联映射】
定义	• 只规定了主存需要映射到Cache中 • 没有规定它映射到cache中的哪一个位置 • 通常使用按内容寻址的相联存储器进行地址映射	• 规定好了主存中每一块都放置在cache中的哪个地方 • 而且相邻块之间映射的位置也是相邻的 • 因为主存的容量肯定比cache大得多，所以其实它相当于一轮轮映射过去	• 先按号分组【如8块Cache分为四组】 • 组内再任意放【组内是全相联映射】
例图			
计算方法	• 全相联中的t,c,b,m和直接映射中的都一样	• 【主存地址长度】主存中存储单元个数为 2^{10},则主存地址长度就是10 • 【Cache地址长度】Cache中存储单元个数为 2^{10},则Cache地址长度就是10 • 【Cache行的总位数】 = 标记位数t + 数据位 + 1位有效位 + 1位脏位(回写策略) t：• 别名：【主存区号】【Tag位】【主存字块标记】 • 通过主存区的标记位数就能知道这个cache是属于主存的第几区 • t = 主存地址长度 - Cache地址长度 • t = 主存大小/Cache大小 c：• Cache块的地址位数 • 如有1k个Cache行，则Cache块的地址位数=c=10 m：• 主存块的地址位数 • 如有1k个主存块，则主存块的地址位数=m=10 b：• 块内地址位数 • 如块的大小为32B，按字节编制，块内地址位数=b=5 有效位：每个Cache行一般都有一个有效位，判断Cache是否命中【计算位数一定要加上】 有效位的作用是指出所在Cache行中的信息是否有效 脏位：如果是回写策略，则需要1个脏位【题目有回写策略记得加上】 LRU位：如果有用LRU替换算法，则增加一位LRU位 其他：• m = t + c • m=主存地址长度 - b • 主存地址长度 = t + c + b	2^c：Cache的总块数 2^q：Cache分组个数 【分组个数 = 分块个数/组内块数】 2^r：组内包含的块数 【r=1，每组包含2块，叫二路组相联】 • s=t+r • q=c-r
王道例题	第12题	第7题，第9题，第22题	第13题，第17题
优点	• 映射灵活 • 块冲突概率比较低 • 空间利用率也高	• 简单、成本低、易实现 • 由于物理位置也是相邻的所以地址变换速度快 • 不需要替换算法	
缺点	• 成本高 • 速度慢耗时多	• 映射方式不够灵活 • 空间利用率最低 • 块冲突概率最高	

Cache 存储器
主存储器
m=t+c
标记 字块0 字块1 字块2^c−1
主存地址
主存字块标记 字块内地址
m = t + c 位 b位
1. 直接映射
i = j mod C
t位 Cache存储体
比较器（t位）
有效位=1?
命中 不命中
主存字块标记 Cache字块地址 字块内地址
组 Cache 共Q组，每组内两块（r = 1）
主存字块标记 组地址 字块内地址
s = t + r 位 q = c − r 位 b 位

例题

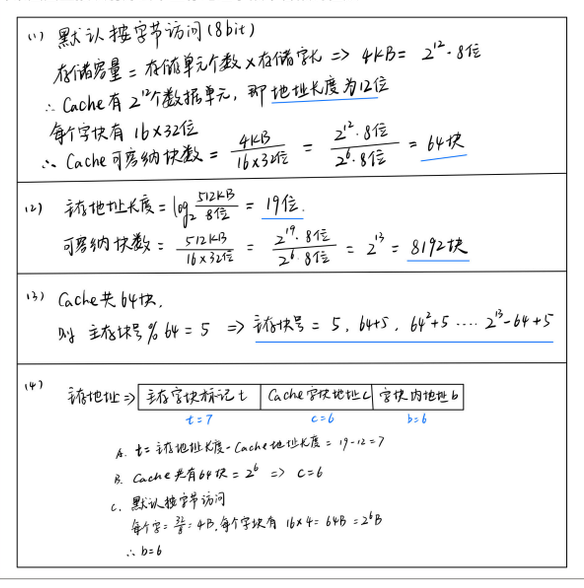
假设主存容量为512KB,Cache容量为4KB,每个字块为16个字，每个字为32位。

(1).Cache地址为多少位？可容纳多少块?

(2).主存地址为多少位？可容纳多少块?

(3).在直接映射方式下，主存的第几块映射到Cache中的第五块（设起始字块号为1）

(4).画出直接映射方式下主存地址字段中各段的位数



(1) 默认按字节访问(8bit)

存储容量 = 存储单元个数 × 存储字长 ⇒ 4KB = 2^{12}·8位

∴ Cache有2^{12}个数据单元，即地址长度为12位

每个字块有 16×32位

∴ Cache可容纳块数 = $\frac{4KB}{16×32位} = \frac{2^{12}·8位}{2^6·8位}$ = 64块

(2) 主存地址长度 = $\log_2 \frac{512KB}{8位}$ = 19位

可容纳块数 = $\frac{512KB}{16×32位} = \frac{2^{19}·8位}{2^6·8位} = 2^{13}$ = 8192块

(3) Cache共64块，则 主存块号 % 64 = 5 ⇒ 主存块号 = 5，64+5，64^2+5 …. 2^{13}−64+5

(4) 主存地址 ⇒ | 主存字块标记 t | Cache字块地址 c | 字块内地址 b | t=7 c=6 b=6

A. t = 主存地址长度 − Cache地址长度 = 19−12 = 7

B. Cache共有64块 = 2^6 ⇒ c=6

C. 默认按字节访问 每个字 = 32/8 = 4B，每个字块有 16×4 = 64B = 2^6B ∴ b=6

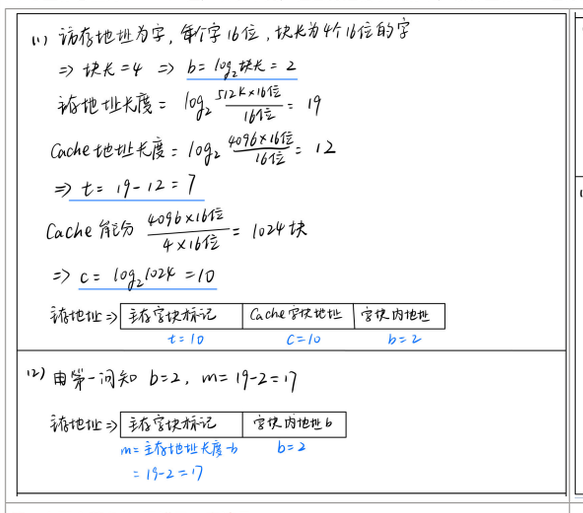
假设主存容量为512K*16位,Cache容量为4096*16位,块长为4个16位的字，访存地址为字。

(1).在直接映射下，设计主存的地址格式。

(2).在全相联映射下，设计主存的地址格式。

(3).在二路组相联映射方式下，设计主存的地址格式。

(4).若主存容量为512K*32位，块长不变，在四路组相联映射下，设计主存的地址格式。



(1) 访存地址为字，每个字16位，块长为4个16位的字

⇒ 块长 = 4 ⇒ b = $\log_2$块长 = 2

主存地址长度 = $\log_2 \frac{512K×16位}{16位}$ = 19

Cache地址长度 = $\log_2 \frac{4096×16位}{16位}$ = 12

⇒ t = 19 − 12 = 7

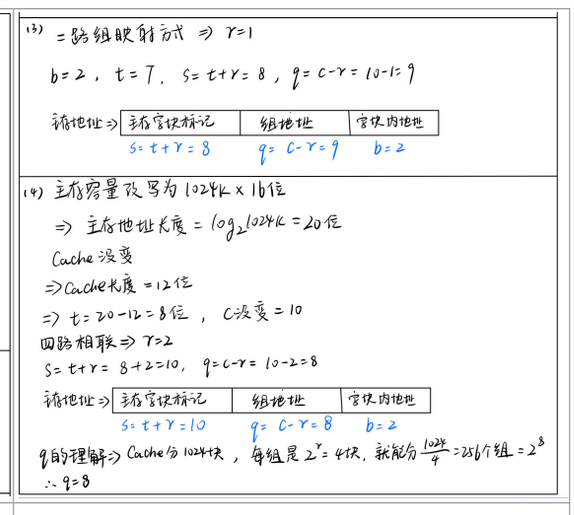
Cache 能分 $\frac{4096×16位}{4×16位}$ = 1024块

⇒ c = $\log_2 1024$ = 10

主存地址 ⇒ | 主存字块标记 | Cache字块地址 | 字块内地址 | t=10 C=10 b=2

(2) 由第一问知 b=2，m = 19−2 = 17

主存地址 ⇒ | 主存字块标记 | 字块内地址 b | m = 主存地址长度为 = 19−2 = 17 b=2



(3) 二路组映射方式 ⇒ r=1

b=2，t=7，s = t+r = 8，q = c−r = 10−1 = 9

主存地址 ⇒ | 主存字块标记 | 组地址 | 字块内地址 | s=t+r=8 q=c−r=9 b=2

(4) 主存容量改写为 1024K × 16位

⇒ 主存地址长度 = $\log_2 1024K$ = 20位

Cache 没变

⇒ Cache长度 = 12位

⇒ t = 20−12 = 8位，C没变 = 10

四路相联 ⇒ r=2

s = t+r = 8+2 = 10，q = c−r = 10−2 = 8

主存地址 ⇒ | 主存字块标记 | 组地址 | 字块内地址 | s=t+r=10 q=c−r=8 b=2

q的理解 ⇒ Cache分1024块，每组是2^r = 4块，就能分 $\frac{1024}{4}$ = 256个组 = 2^8 ∴ q=8

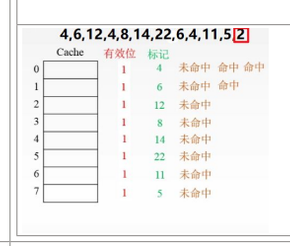
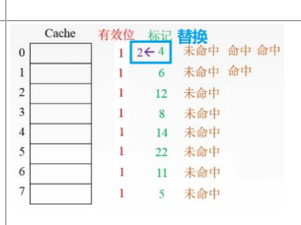
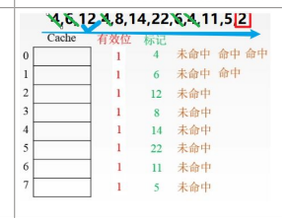
第一个那个蓝色的t写错了，应该是t=7个

【替换算法】解决Cache满了如何处理的问题

随机算法	• 啥都不管只要你想映射的这块cache有内容了，就直接覆盖替换它 • 即使原先在这块cache上的数据可能经常会用到，可能刚刚用完，也没关系，替换了就是了 • 现实中不会以这种方式设计
先进先出FIFO	• 按调入cache的先后顺序来淘汰，先进的先替换 • 需要记录进入cache的先后次序 • 实现起来比较简单 • 缺点就是只考虑了时间层面的问题，会把一些经常使用的也当作最早进入cache的块给替换掉
近期最少使用LRU	• 把最近比较少用的替换掉 • 需要记录进入cache的先后次序 • 也需要软件计数器来记录使用的频率 • 实现起来是比较复杂且开销大 • 根据程序访问局部性原理选择近期使用得最少的存储块作为替换的块
最不经常使用LFU	• 只统计使用次数，用的最少的就替换掉谁 • 通常也是需要硬件设计计数器支持

例题

设Cache由8个块构成，CPU依次访问的主存地址块号为：4，6，12，4，8，14，22，6，4，11，5，2(十进制)，求：
1）假设地址映射方式为全相联映射，在采用FIFO、LRU、LFU替换算法时，分别求Cache命中次数。

	先进先出（FIFO）算法	近期最少使用（LRU）算法	最不经常使用（LFU）算法
• 有效位表示当前cache块是否存有数据 • 标记位记录存入的数据是几	• 由于最先进入的是0号cache • 因此0号cache替换掉，标记位标记为2	• 从后往前，先把前面使用过的划去 • 没有被划去的意味着用的比较少 • 然后再从前往后看 • 看第一块没有被划去的就决定替换它	• 统计使用次数 • 4号块用了3次，6号块用了2次 • 其他几块均用了1次，需要更多判断依据 • 最终替换掉Cache第2块（12号
			

4,6,12,4,8,14,22,6,4,11,5,2
Cache
有效位
标记
替换
未命中
命中

王道例题：第18题【LRU】，第20题【计算缺失率】

【写入/更新策略】

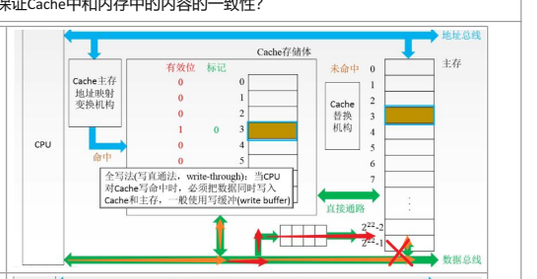
什么是更新策略?

- 更新策略要解决的问题：如果cpu要向内存写入数据，我们如何保证Cache中和内存中的内容的一致性?

Cache写命中

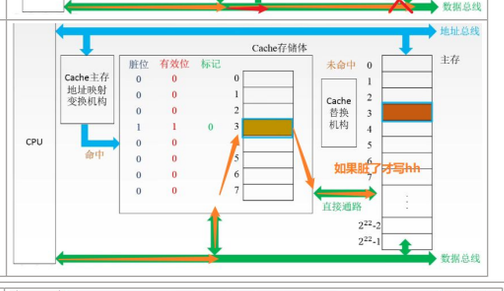
全写法/写直达法

- 在写的时候cpu将数据通过数据总线橙色箭头方向传输
- 由于cpu写给cache的速度和写入主存的速度会差很多
- 所以需要设计一个缓冲，先写到缓冲块中，再慢慢写入


地址总线
Cache存储体
Cache主存地址映射变换机构
有效位 标记
未命中
Cache替换机构
主存
CPU
命中
全写法(写直通法，write-through)：当CPU对Cache写命中时，必须把数据同时写入Cache和主存，一般使用写缓冲(write buffer)
直接通路
数据总线

回写法

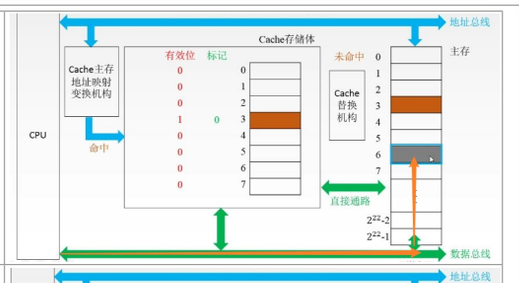
- 在cpu执行写操作时，先按橙色线写入cache存储体中
- 此时并没有修改主存的内容，会在cache中设计一个脏位
- 当一块中的任何一个单元被修改时，脏位(修改位)被置“1”
- 需要替换掉这一块时，如果修改位为“1”，则必须先把这一块写回到主存中，然后才能再调入新的块;
- 如果修改位为“0”，则这一块不必写回主存，只要用新调入的块覆盖这一块即可


脏位 有效位 标记
如果脏了才写hh
直接通路
数据总线

Cache写没命中

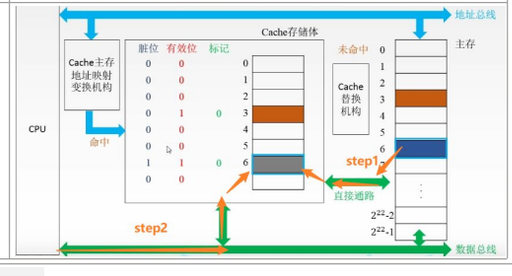
写分配法

- 先将数据从内存中调入到cache中
- 再按回写法修改cache

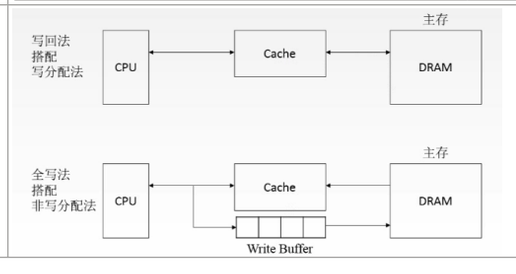


非写分配法

- 直接去内存修改


step1
step2

四种方法的搭配


写回法
搭配
写分配法
CPU
Cache
主存
DRAM
全写法
搭配
非写分配法
Write Buffer